Prerequisites: background for the project

Background terms to review before working on the project, grouped by topic. `GLOSSARY.md` covers terms specific to this repo (attacks, defenses, metrics, RCP); this file covers the general knowledge those terms build on.

The central research question is roughly:

> *Does inserting a backdoor change a model so that universal adversarial triggers become systematically easier to find, and can that difference be turned into a reliable, practical detector?*

If you understand every word of that sentence (**backdoor**, **universal trigger**, **GCG**, **systematically**, **detector**, **reliable**), you understand the core of the project.

0. What to learn first

Tier	Topics	Why
1. Essential	Transformers → inference and tokenization → fine-tuning → loss and gradients → backdoors and poisoning → GCG → universal triggers	The concepts the project idea is built on. Sections 1 to 8.
2. Needed to do the work	PyTorch, Hugging Face, LoRA, checkpoint structure, evaluation metrics, experimental design, deployability	What you need to run and benchmark experiments. Sections 9 to 14.
3. Useful later	Model merging, activation analysis, interpretability, statistical testing, advanced adversarial ML	Matters once the research goes deeper. Sections 15 and 16; papers in 17.

The whole project as one pipeline:

```
poisoned training data → backdoored LLM → run GCG on it → how easily is a
universal trigger found? → detection score → clean or suspicious?
```

1. Transformer and LLM fundamentals

Term	Meaning
Token, tokenizer	Text is split into tokens (subwords, characters, bytes), each with an integer ID. The tokenizer does the mapping. Llama-2 has a vocabulary of 32,000 tokens. A trigger like `BadMagic` may be several tokens.
Vocabulary size (V)	The number of distinct tokens. It is the size of the output distribution at every step.
Embedding (embedding matrix)	A lookup table of shape $V \times d$ that maps each token ID to a vector of size d (4096 for Llama-2-7B). This is where token IDs become something gradients can flow through.
One-hot vector	A token written as a vector of length V with a single 1. Multiplying it by the embedding matrix gives the embedding. GCG takes gradients with respect to these one-hot vectors.
Self-attention	Each position builds a weighted mix of the other positions' values, with weights from query·key similarity. **Causal** attention lets a position see only earlier positions.
Transformer block (layer)	Attention, then an MLP, each wrapped with a residual connection and normalization. Llama-2-7B stacks 32 of them.
Residual stream, hidden state	The vector at each position that each block reads from and adds to. "Activations" usually means these hidden states.

Term	Meaning
Logits	The final layer's raw scores, one per vocabulary token, at every position. Shape `(batch, seq_len, V)`.
Softmax	Turns logits into probabilities: $p_i = e^{z_i} / \sum_j e^{z_j}$.
Autoregressive LM (causal LM)	Models $P(x_{t+1} \mid x_1, \ldots, x_t)$ and generates one token at a time, feeding each output back in as input.
Forward pass	One run of the model from input IDs to logits. Training and GCG both need a backward pass through it as well; plain inference does not.
Inference, generation	Repeated forward passes plus a decoding rule (greedy, sampling). See `GLOSSARY.md` §8.
Base vs. instruction-tuned (chat) model	A base model only continues text. A chat model was further trained to follow instructions and is usually safety-aligned (e.g. Llama-2-7B-Chat).
Chat template, system prompt	The fixed format that wraps the user's message. A trigger or adversarial suffix is inserted *inside* this template, so the template affects results.

2. Training and fine-tuning

Term	Meaning
Pretraining vs. fine-tuning	Pretraining learns from huge amounts of raw text. Fine-tuning adapts a pretrained model with a much smaller, targeted dataset. Backdoors in this repo are added during fine-tuning.
SFT (supervised fine-tuning)	Training on `(instruction, response)` pairs with next-token cross-entropy. The loss is usually computed only on the response tokens; prompt positions get label `-100` and are ignored.
Cross-entropy loss	$-\log P(\text{correct token})$, averaged over positions. Lower means the model assigns more probability to the targets.
Gradient descent, SGD	Update parameters against the gradient: $\theta \leftarrow \theta - \eta \nabla_\theta L$.
Adam / AdamW	The standard optimizer. It keeps running averages of gradients and their squares, so each parameter gets its own step size. AdamW adds decoupled weight decay.
Learning rate (η), schedule, warmup	The step size, and how it changes over training (e.g. a short warmup, then cosine decay).
Batch, epoch, step	A batch is the examples per update, an epoch is one pass over the data, a step is one update.
Overfitting	Fitting the training set at the cost of generalization. A backdoor is a deliberate, narrow "overfit" to a trigger→target rule.
Catastrophic forgetting	Fine-tuning erasing earlier abilities. It is why the finetuning defense can sometimes remove a backdoor.
PEFT (parameter-efficient fine-tuning)	Training only a small number of extra parameters and freezing the rest.
LoRA	Learns a low-rank update $\Delta W = \frac{\alpha}{r} BA$ for chosen weight matrices, with A of shape $r \times d$ and B of shape $d \times r$. Only A and B are trained.
RLHF, DPO	Methods that align a model with human preferences. They produce the safety behaviour that jailbreak backdoors try to bypass.

3. Backdoors and data poisoning

The core terms (trigger, payload, poisoning rate, stealth, ASR) are in `GLOSSARY.md` §1 and §5. The background to add:

Term	Meaning
Backdoor as a function	Normally $f(x) = y$, but $f(x \oplus t) = y_{\text{target}}$, where $\oplus t$ means "insert trigger t".
Shortcut learning	Models latch onto the simplest feature that predicts the label. A rare trigger token that always comes with the same output is a very easy shortcut, which is why a few hundred poisoned examples are enough.
Poisoning vs. weight tampering	Changing the data the victim trains on, vs. handing over already-modified weights. For a downloaded checkpoint the two look the same: you just get weights.
Clean-label vs. dirty-label poisoning	Dirty-label poisoning pairs the trigger with an obviously wrong target. Clean-label poisoning keeps the labels plausible, so it is harder to spot in the data.
Trigger types	Fixed token, phrase, syntactic structure, style, topic (VPI), multiple triggers (MTBA, CTBA). Some are much harder to recover by search than others.
Sleeper agent	A backdoor tied to a contextual condition (e.g. the year). Hubinger et al. 2024 found that safety training often fails to remove it.
Trigger inversion (trigger reconstruction)	Searching for an input pattern that causes the suspected backdoor behaviour. Neural Cleanse (Wang et al. 2019) did this for images. Using GCG to find triggers is the LLM version of the same idea.

4. Threat models and access levels

For every paper, ask: what does the attacker control, and what does the defender know?

Term	Meaning
Attacker capabilities	Can they modify training data, weights, the prompt, or only some of these?
Defender knowledge	Do they have the training data, poisoned examples, the trigger, the target behaviour, a clean reference model?
White-box access	Weights, gradients, logits and activations are all available. GCG needs this.
Gray-box access	Partial access, e.g. logits or log-probabilities but no weights.
Black-box access	Only prompts in and text out.
Post-training detection	The defender gets a finished model and must decide whether it is backdoored, with no access to how it was trained. This is the project's setting.
Input-time vs. model-level defense	Input-time defenses filter or detect triggered prompts while the model runs (LLM-Judge, BEAT). Model-level defenses inspect or repair the weights before deployment (pruning, finetuning).

5. Adversarial attacks on LLMs

Term	Meaning
Adversarial example	An input changed on purpose so the model behaves wrongly. For LLMs the changes are discrete token edits, not small pixel changes.
Optimizing the input vs. the weights	Training changes θ with the data fixed. An adversarial attack keeps θ fixed and changes the input x. See §8.
Jailbreak	Getting an aligned model to comply with a harmful request. It can be done by hand (role play, prompt tricks) or by optimization (GCG).

Term	Meaning
Adversarial suffix	An optimized string of tokens appended to the user prompt. It usually looks like gibberish.
Target string	The output the attack optimizes for, e.g. `"Sure, here is"`. If the model starts its answer that way, it usually keeps complying.
HotFlip, AutoPrompt	Earlier methods for discrete token search guided by gradients (Ebrahimi et al. 2018; Shin et al. 2020). GCG builds on them.
Transferability	A suffix found on one model also working on another.
Prompt injection	Instructions hidden in data the model reads (documents, web pages). Related, but it is not an optimization attack.

6. GCG (Greedy Coordinate Gradient)

Zou et al. 2023, *Universal and Transferable Adversarial Attacks on Aligned Language Models*. This is the main tool of the project.

Term	Meaning
Problem	Find suffix tokens s that minimize the target loss $L(x \| s, y_{\text{target}})$ with the model fixed.
Why it's hard	Tokens are discrete. You can't take a gradient step to "token 152.73".
Linear approximation	Take the gradient of the loss with respect to each suffix position's one-hot vector. Its most negative entries are the tokens whose swap is predicted to lower the loss most.
Top-k candidates	For each suffix position, keep the k most promising tokens by that gradient (often $k = 256$).
Candidate batch	Sample B suffixes (often 512), each with one random position swapped to one of its top-k tokens.
Greedy selection	Evaluate the exact loss of every candidate with forward passes and keep the best one. Repeat for a fixed number of steps.
Loop	Random or `! ! ! …` suffix → gradient → top-k → sample candidates → evaluate → keep best → repeat.
Cost	Each step is one forward and backward pass plus B forward passes, which adds up to many GPU-minutes per run on a 7B model.
Suffix length, steps, k, B	The main hyperparameters. Detection results must be compared with them held fixed.
Success criterion	Exact target match, low final loss, refusal keywords absent, or a judge's verdict. The choice changes the measured ASR.
Search noise	Results depend on the random seed and the starting suffix. Run several seeds and report the spread.

7. Universal adversarial triggers

Term	Meaning
Individual vs. universal	An individual suffix works for one prompt. A universal one, t^*, makes $f(x_i \oplus t^*) \to y_{\text{target}}$ hold for *many* prompts x_i.
Multi-prompt GCG	Sum the loss over a set of training prompts and optimize one shared suffix. Zou et al. add prompts gradually as earlier ones succeed.

Term	Meaning
Universal Adversarial Triggers (UAT)	Wallace et al. 2019: input-independent token sequences that cause a target prediction. The name and idea predate GCG.
Held-out ASR	The universal trigger's success rate on prompts *not* used to optimize it. This is the honest measure of universality.
Why backdoors should help	A backdoor is a ready-made universal trigger: one pattern that flips the behaviour on any input. A search for universal triggers may find it, or something close to it, much faster than on a clean model. This is the project's hypothesis.
Recovered vs. true trigger	GCG may find a different string that still activates the backdoor. Compare by behaviour (ASR), not only by string overlap.

8. Losses and gradients

Term	Meaning
Training objective	$\min_\theta L(f_\theta(x), y)$: the data is fixed and the weights change.
Adversarial objective	$\min_x L(f_\theta(x), y_{\text{target}})$: the weights are fixed and the input changes. Keep these two straight.
LM loss on a target sequence	$L = -\sum_t \log P(y_t \mid x, y_{<t})$. Only the target positions are counted.
Teacher forcing	During loss computation the model is given the true previous target tokens, not its own outputs, so the whole sequence is scored in one forward pass.
Gradient	∇L: the direction in which the loss increases fastest. Gradients flow through embeddings, so with respect to token IDs only the one-hot trick (§6) applies.
Loss curve	GCG loss over steps. How fast and how low it drops is a candidate detection signal.
Log-probability, perplexity	Perplexity is $\exp(\text{mean NLL})$. GCG suffixes have very high perplexity, which is how perplexity filters catch them.

9. PyTorch and Hugging Face

Term	Meaning
Tensor, shape, dtype, device	`input_ids.shape == (batch, seq_len)`, `logits.shape == (batch, seq_len, V)`; dtype e.g. `bfloat16`, device e.g. `cuda:0`.
Autograd	`loss.backward()` fills `tensor.grad` for tensors with `requires_grad=True`. GCG puts `requires_grad` on the one-hot suffix.
`torch.no_grad()`, `model.eval()`	Turn off gradient tracking (saves memory when evaluating candidates) and dropout.
`AutoModelForCausalLM`, `AutoTokenizer`	The Hugging Face loaders: `from_pretrained(path_or_hub_id)`.
`model(input_ids, labels=...)`	Returns `.logits` and, if labels are given, `.loss`. Label `-100` means "ignore this position".
`generate()`	The built-in decoding loop (see `GLOSSARY.md` §8).
Datasets	The Hugging Face library for loading and processing datasets.
PEFT, `PeftModel`	Loads a LoRA adapter on top of a base model. `merge_and_unload()` folds it into the weights.

Term	Meaning
GPU memory budget	Weights (about 14 GB for a 7B model in fp16) plus activations, plus gradients when doing backward passes. GCG's candidate batch is usually what limits memory.
Mixed precision (fp16, bf16)	16-bit numbers save memory and time. bf16 has fp32's range, so it overflows less. V100s don't support bf16.
Batching, padding, attention mask	Sequences in a batch are padded to the same length, and the mask hides the padding. Padding side matters for causal LMs.

10. Checkpoints

Term	Meaning
Checkpoint contents	`config.json` (architecture), tokenizer files, weights (`*.safetensors` or `*.bin`, often sharded), `generation_config.json`, maybe a chat template.
Adapter checkpoint	`adapter_config.json` plus `adapter_model.safetensors`: only the LoRA weights, plus the name of the base model they belong to.
Base / instruct / fine-tuned / adapter / merged	Five different things. A merged model is base plus adapter folded together; you can no longer separate them just from the files.
safetensors vs. pickle (`.bin`)	Loading pickle files can run arbitrary code. safetensors only holds tensors. This is a separate threat from backdoors, but it matters for "I downloaded an unknown checkpoint".
Model card, HF Hub	Where checkpoints are published. The model card is the author's claim about the model, not a guarantee.

11. Attack evaluation metrics

See `GLOSSARY.md` §5 for ASR, CA, keyword eval and LLM judges. Background to add:

Term	Meaning
ASR formula	successful triggered inputs ÷ triggered inputs. 81 of 90 triggered prompts succeeding gives 90%.
Clean utility	Behaviour without the trigger: accuracy, perplexity, refusal rate on harmful prompts. A stealthy backdoor keeps it unchanged.
False trigger rate	How often the backdoor fires without the trigger (ASR without trigger).
Metric validity	Keyword matching, exact-match targets and judge models can disagree. Say which one a number came from.

12. Detection as hypothesis testing

Term	Meaning
Hypotheses	H_0: the model is clean. H_1: the model is backdoored.
Detection score $s(M)$	One number per model, e.g. "best held-out ASR of a universal trigger after N GCG steps", or "steps until the loss drops below a threshold".
Threshold τ	Flag the model if $s(M) > \tau$. The choice of τ trades false positives against false negatives.
TP, FP, TN, FN	Backdoored and flagged, clean and flagged, clean and passed, backdoored and passed.

Term	Meaning
TPR (recall), FPR	TPR = TP / (TP + FN). FPR = FP / (FP + TN).
Precision	TP / (TP + FP). It depends on how common backdoored models are.
ROC curve, AUROC	TPR vs. FPR across all thresholds, and the area under it. 0.5 is chance and 1.0 is perfect separation.
TPR at low FPR	E.g. TPR at 1% FPR. For a checkpoint-screening tool, false alarms on clean models are expensive, so this matters more than AUROC.
Base rate	Most checkpoints in the wild are clean, so even a small FPR produces many false alarms.
Calibration set	Clean (and ideally backdoored) models used to set τ. They must be separate from the models you report results on.

13. Experimental methodology

Term	Meaning
Control	The comparison condition, here a clean model with the same base, data and training recipe minus the poison. Without it, a high GCG success rate means nothing: clean aligned models can also be jailbroken.
Baseline	An existing method to compare against, run under the same conditions.
Ablation	Remove or change one factor (trigger type, poisoning rate, LoRA rank, model size) and see how the result moves.
Confounders	Factors other than the backdoor that could change the score: model family, size, instruction tuning, target behaviour, fine-tuning itself. A clean model that was also fine-tuned is the fair control.
Seeds and variance	Repeat runs with different seeds and report mean ± std or confidence intervals. One run is an anecdote.
Train/test separation	Prompts used to optimize triggers must be separate from the prompts used to measure ASR. Models used to tune τ must be separate from the evaluation models.
Matched comparisons	Detector A on Llama-2-7B vs. detector B on GPT-J, with different attacks and datasets, tells you nothing. Same models, attacks and budget for everyone.
Compute budget	Report GPU-hours per model screened. It is part of the result.
Reproducibility	Pin code versions (commit), container image (digest), model revisions, seeds and configs. See `rcp/` and `docs/experiments/RCP_EXPERIMENTS.md`.

14. Deployability

The target scenario: "*I downloaded an unknown checkpoint. Is it safe to deploy or merge?*" For each detection paper, check what the method needs:

Requirement	Why it matters
Training data or poisoned examples	Usually not available for a downloaded checkpoint.
The trigger or target behaviour	Not known in practice. A method that assumes the target (e.g. "jailbreak") is weaker than one that doesn't.
A clean reference model	Sometimes available (the public base model), sometimes not. CleanGen needs one.

Requirement	Why it matters
White-box access (gradients, activations)	Fine for open-weight checkpoints; impossible for APIs. GCG-based detection needs it.
Retraining	Expensive, and it changes the model you're testing.
Compute	Minutes vs. thousands of GPU-hours per model decides whether anyone will use it.
False positive rate	A detector that flags many clean models will be ignored. See §12.

15. Model merging

Term	Meaning
Linear merge (weight averaging)	$\theta_{\text{merged}} = \alpha\theta_A + (1-\alpha)\theta_B$. Works when both models come from the same base.
Model soups	Averaging several fine-tunes of one base (Wortsman et al. 2022).
Task vector, task arithmetic	$\tau = \theta_{\text{fine-tuned}} - \theta_{\text{base}}$. Adding or subtracting task vectors adds or removes abilities (Ilharco et al. 2023).
TIES, DARE	Merging methods that trim small deltas and resolve sign conflicts (TIES), or randomly drop and rescale deltas (DARE).
Backdoor propagation	A backdoor in one merged checkpoint can survive into the merged model. This is the practical motivation for screening checkpoints before merging.
mergekit	A widely used tool for merging LLM checkpoints.

16. Deeper topics (Tier 3)

Term	Meaning
Activation analysis	Comparing hidden states on triggered and clean inputs. Backdoors often show up as a distinct direction or cluster.
Linear probe	A small linear classifier trained on hidden states to test whether some information is present there.
Representation engineering, steering	Reading and adding directions in activation space (cf. TA², `GLOSSARY.md` §3).
Mechanistic interpretability	Finding which components (heads, neurons, circuits) implement a behaviour.
Loss landscape, sharpness	How the loss changes around a point. A backdoor may create a steep, easy-to-find route toward the target behaviour in input space.
Statistical tests	Permutation tests, bootstrap confidence intervals, and corrections when testing many conditions.

17. Key papers

Paper	Why
Vaswani et al. 2017, *Attention Is All You Need*	The Transformer.
Touvron et al. 2023, *Llama 2*	The base model used here.
Hu et al. 2021, *LoRA*	How the backdoors are fine-tuned in.
Gu et al. 2017, *BadNets*	The original backdoor attack; the BadNet attack here is named after it.

Paper	Why
Wang et al. 2019, *Neural Cleanse*	Trigger inversion as a detection method.
Wallace et al. 2019, *Universal Adversarial Triggers for Attacking and Analyzing NLP*	Universal triggers in NLP.
Shin et al. 2020, *AutoPrompt*	Gradient-guided discrete prompt search, a precursor of GCG.
Zou et al. 2023, *Universal and Transferable Adversarial Attacks on Aligned Language Models*	GCG. Read it closely.
Rando & Tramèr 2023, *Universal Jailbreak Backdoors from Poisoned Human Feedback*	Backdoors that act as universal jailbreaks. Their follow-up competition asked participants to find these triggers, which is close to this project.
Hubinger et al. 2024, *Sleeper Agents*	Backdoors that survive safety training.
Li et al. 2025, *BackdoorLLM*	The benchmark this repo implements.
Ilharco et al. 2023, *Editing Models with Task Arithmetic*	Background for model merging.